FICHE TECHNIQUE | DÉVELOPPEMENT WEB ET MOBILE

Voyagéo

Fiche technique

Base de données, paiement, e-mail et mise en ligne : où brancher chaque élément du site

PROJET	Voyagéo : site web de réservation de tickets de bus au Bénin
NATURE DU DOCUMENT	Fiche technique d'intégration (développement et base de données)
DESTINATAIRES	Développeur et responsable de la base de données
TECHNOLOGIES	HTML5, CSS3, Bootstrap 5, JavaScript, Node.js, Express.js
ÉLÈVE (DÉVELOPPEMENT)	Caleb
CLASSE	Première DWM
ÉTABLISSEMENT	HTC-CETIP
ANNÉE SCOLAIRE	2026-2027
DATE	10 octobre 2026

Aucun mot de passe ni aucune clé secrète n'est écrit dans ce document : tout se trouve dans le fichier .env.

Sommaire

1. Vue d’ensemble et répartition du travail . . . 3

2. Démarrage rapide . . . 3

3. Le fichier .env : tous les branchements . . . 4

4. Base de données : où et comment la brancher . . . 5

5. Paiement en ligne : où mettre les clés . . . 7

6. E-mail de confirmation (optionnel) . . . 8

7. Routes de l’API . . . 9

8. Sécurité . . . 10

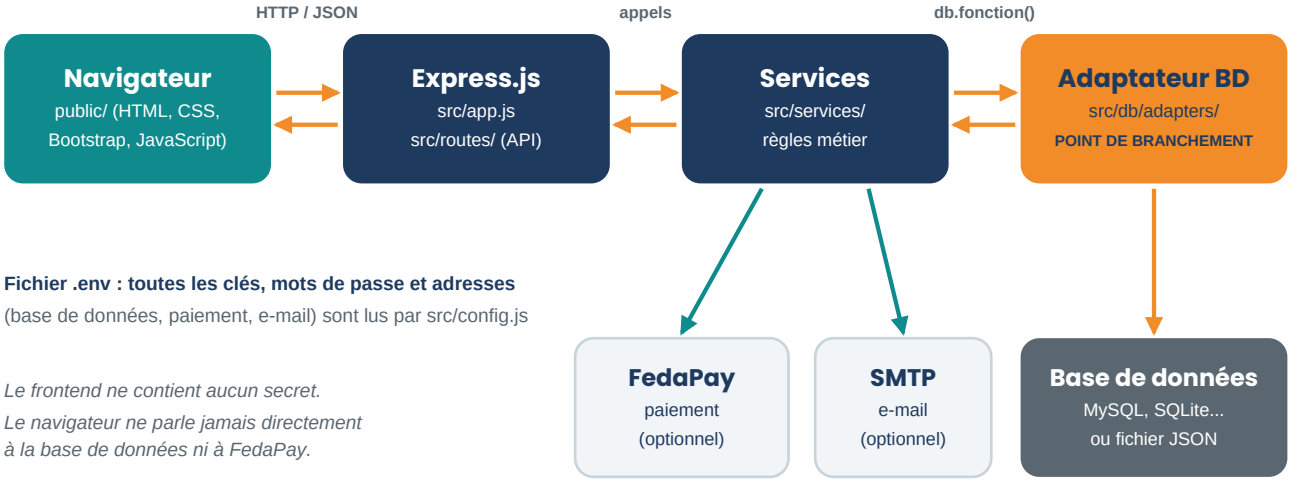
9. Mise en ligne : liste de contrôle . . . 10

10. Tests et dépannage . . . 11

À qui s’adresse cette fiche ? Au développeur du site (Caleb) et au responsable de la base de données (son collègue). Elle indique, point par point, **quel fichier modifier** et **où mettre chaque clé ou mot de passe**. Le contrat détaillé de la base de données est aussi fourni dans `docs/CONTRAT_BASE_DE_DONNEES.md`.

1. Vue d'ensemble et répartition du travail

Le site suit l'architecture du cahier des charges : un navigateur qui parle à un serveur Node.js/Express en JSON, et un serveur qui accède aux données uniquement par une **couche d'accès à la base de données**. Le site n'écrit jamais de SQL en dehors de cette couche : le collègue peut donc choisir et brancher la base de données sans toucher au reste.



Élément	Dossier / fichier	Qui ?	Rôle
Frontend	`public/`	Développeur	Pages HTML, feuille de style de la charte graphique, scripts JavaScript.
Backend	`src/routes/`, `src/services/`, `src/app.js`	Développeur	API, règles de réservation, sécurité, paiement, e-mail.
Branchement base de données	`src/db/`	Responsable BD	Contrat (`contract.js`), choix de l'adaptateur (`index.js`), adaptateurs.
Schéma de la base	`database/schema.sql`	Responsable BD	Tables, clés primaires et étrangères, règle anti-double réservation.
Configuration	`.env`	Les deux	Adresses, mots de passe et clés. Jamais sur GitHub.

2. Démarrage rapide

Étape	Commande ou action	Résultat
1	Installer Node.js 20.12 ou plus récent (version LTS).	Commande `node -v` disponible.
2	`npm install`	Télécharge Express, Bootstrap, etc. dans `node_modules/` (connexion Internet nécessaire).
3	Vérifier le fichier `.env` (fourni, déjà rempli pour le test).	Le site utilise la base « json » : aucune installation de base nécessaire.
4	`npm start`	Site disponible sur http://localhost:3000. Le compte administrateur du `.env` et des données de démonstration sont créés.
5	`npm test`	Lance 23 tests automatiques (contrat base de données, API, paiement).

Le mot de passe administrateur se change ensuite dans le site (menu « Mon compte »). En cas d'oubli : modifier `ADMIN_PASSWORD` dans `.env` puis lancer `npm run admin:reset`.

3. Le fichier .env : tous les branchements

Le fichier `.env` (à la racine du projet) est le seul endroit où l'on met les valeurs qui changent d'un ordinateur ou d'un serveur à l'autre. Le modèle commenté est `.env.example`. **Règle d'or :** une valeur qui contient le caractère # doit être écrite entre guillemets (exemple : `ADMIN_PASSWORD="abc#123"`), sinon tout ce qui suit le # est ignoré.

Variable	Rôle	Valeurs / exemple
NODE_ENV	Mode du serveur.	`development` (tests) ou `production` (site en ligne).
PORT	Port d'écoute.	`3000`
APP_BASE_URL	Adresse publique du site, sans « / » final. Utilisée dans les QR codes, les e-mails et le retour de paiement.	`http://localhost:3000` puis `https://www.mon-site.bj`
SESSION_SECRET	Phrase secrète qui protège les sessions. Obligatoire en production.	Longue phrase aléatoire (32 caractères minimum).
TRUST_PROXY	À mettre à `true` derrière un hébergeur ou un proxy (HTTPS).	`false` en local.
ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NOM, ADMIN_PRENOM	Compte administrateur créé au premier démarrage s'il n'existe pas. Jamais écrasé ensuite.	Voir le fichier `.env`.
DB_DRIVER	Adaptateur de base de données.	`json` | `mysql` | `custom`
JSON_DB_FILE	Fichier de la base « json » (tests).	`./data/db.json`
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, DB_CONNECTION_LIMIT	Connexion MySQL/MariaDB (adaptateur `mysql`).	`localhost`, `3306`, compte dédié, mot de passe, `voyageo`, `10`
DEMO_DATA	Crée des compagnies, bus et départs fictifs si la base est vide.	`true` pour la démonstration, `false` en production.
ANNULATION_HEURES	Délai minimal avant le départ pour qu'un voyageur annule.	`24`
RESERVATION_ATTENTE_MINUTES	Durée de blocage d'une place en attente de paiement.	`15`
CLOTURE_RESERVATION_MINUTES	Réservations fermées X minutes avant le départ.	`30`
TIMEZONE_OFFSET	Décalage horaire du Bénin.	`+01:00`
PAYMENT_MODE	Mode de paiement (voir partie 5).	`off` | `simulation` | `fedapay`
FEDAPAY_ENV, FEDAPAY_PUBLIC_KEY, FEDAPAY_SECRET_KEY	Clés FedaPay.	`sandbox` puis `live` ; clés du tableau de bord FedaPay.
KKIAPAY_PUBLIC_KEY, KKIAPAY_PRIVATE_KEY, KKIAPAY_SECRET	Emplacement réservé (KkiaPay non intégré).	Vide.
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, MAIL_FROM	Envoi des e-mails (voir partie 6). Vide = e-mails désactivés.	`smtp.gmail.com`, `587`, `false`

4. Base de données : où et comment la brancher

4.1 Principe

Tout le site appelle des fonctions du type `db.createReservation(...)` ou `db.searchHoraires(...)`. Ces fonctions sont fournies par un **adaptateur**. Le fichier `src/db/index.js` charge l'adaptateur choisi par `DB_DRIVER` dans le `.env` :

DB_DRIVER	Fichier	Usage
`json`	`src/db/adapters/json.js`	Base dans un fichier. Complet et testé. Sert au développement, à la démonstration et d'exemple de comportement.
`mysql`	`src/db/adapters/mysql.js` + `database/schema.sql`	Proposition pour MySQL/MariaDB. **Non testée sur un vrai serveur par son auteur : à relire et à valider.**
`custom`	`src/db/adapters/custom.js`	Votre propre adaptateur (SQLite, PostgreSQL, MongoDB...). Partir de `_template.js` (à copier en `custom.js`).

4.2 Les 4 points de branchement

#	Où	Que faire
1	`.env`	Choisir `DB_DRIVER` et renseigner `DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME`.
2	`database/schema.sql`	Créer la base et les tables (à exécuter dans MySQL ou phpMyAdmin). Créer un compte dédié qui n'a que `SELECT, INSERT, UPDATE, DELETE`.
3	`src/db/adapters/`	Valider `mysql.js` ou écrire `custom.js` en respectant le contrat (liste en 4.5).
4	`src/db/errors.js`	Lancer `new DbError('SEAT_TAKEN')` (et les autres codes) dans les cas prévus par le contrat.

4.3 Checklist du responsable de la base

- Installer MySQL ou MariaDB (ou XAMPP) et exécuter `database/schema.sql`.
- Remplir les lignes `DB_*` du `.env` et mettre `DB_DRIVER=mysql`.
- `npm install` puis `npm run check:db` : vérifie que toutes les fonctions existent et que la connexion fonctionne (lecture seule).
- Sur une **base de test vide** (jamais la vraie) : `CONTRACT_TEST_USE_ENV=1 npm test`. Les 12 tests du contrat doivent passer, dont « 10 demandes simultanées pour la même place = 1 seule réussite ».
- `npm start` : le compte administrateur et les données de démonstration sont créés automatiquement (via le contrat, donc avec n'importe quel adaptateur).

4.4 Règles critiques

Règle	Pourquoi
Une place = un seul voyageur. `createReservation` est atomique : transaction + verrou du voyage + clé unique `(id_horaire, place_active)`.	Sinon deux voyageurs peuvent réserver la même place. C'est le risque n°1 du cahier des charges.
`place_active` = numéro de place si la réservation est active, `NULL` si elle est annulée.	Libère la place à l'annulation, tout en gardant l'historique.
Dates en texte : `AAAA-MM-JJ` ; heures `HH:MM` ; dates-heures en UTC au format ISO 8601.	Le site et la base doivent parler le même format, sans conversion de fuseau cachée.
Mot de passe : stocker uniquement `mot_de_passe_hache` (déjà haché par le site, format scrypt).	Jamais de mot de passe en clair en base. Colonne de 255 caractères.
Requêtes préparées (?) uniquement, jamais de texte collé dans le SQL.	Protège contre l'injection SQL.
Comparaison des villes insensible à la casse et aux accents (collation `utf8mb4_unicode_ci`).	« cotonou » doit trouver « Cotonou ».
Suppression d'un élément utilisé : lancer `IN_USE`.	Le site affiche un message clair au lieu d'une erreur.

4.5 Tables du schéma proposé

Table	Clé primaire	Clés étrangères	Colonnes principales
compagnie	id_compagnie	—	nom (unique), telephone, email, adresse, statut
utilisateur	id_utilisateur	id_compagnie (gestionnaire)	nom, prenom, email (unique), telephone, mot_de_passe_hache, role, statut, date_creation
bus	id_bus	id_compagnie	immatriculation (unique), nombre_places (1 à 80), type
trajet	id_trajet	id_compagnie	ville_depart, ville_arrivee, duree_estimee (minutes)
horaire	id_horaire	id_trajet, id_bus	date_depart, heure_depart, heure_arrivee, tarif_fcfa, statut
reservation	id_reservation	id_utilisateur, id_horaire	numero_ticket (unique), numero_place, place_active, nom/prenom/telephone_passager, piece_identite, statut, date_reservation, date_annulation. **Unique (id_horaire, place_active)**
paiement	id_paiement	id_reservation	montant_fcfa, mode, reference_transaction (unique), statut, date_paiement

4.6 Fonctions que l'adaptateur doit fournir (contrat)

Toutes sont `async`. « Enrichi » signifie : l'objet contient aussi les informations jointes (voir `docs/CONTRAT_BASE_DE_DONNEES.md`). `update` retourne l'objet mis à jour ou `null` ; `delete` retourne `true` ou `false`.

Fonction	Rôle	Retour / erreurs
init, close, ping	Connexion au démarrage, fermeture, test de santé.	—
findUserByEmail, findUserById	Retrouver un compte (e-mail insensible à la casse).	utilisateur ou null
createUser, updateUser, listUsers	Créer, modifier, lister les comptes.	DUPLICATE_EMAIL
listCompagnies, getCompagnie, createCompagnie, updateCompagnie, deleteCompagnie	Gestion des compagnies.	IN_USE à la suppression
listBus, getBus, createBus, updateBus, deleteBus	Gestion des bus.	DUPLICATE_PLATE, IN_USE
listTrajets, getTrajet, createTrajet, updateTrajet, deleteTrajet	Gestion des liaisons entre villes.	IN_USE
searchHoraires({ville_depart, ville_arrivee, date})	Recherche publique des voyages « programmés ».	horaires enrichis, triés par heure

Fonction	Rôle	Retour / erreurs
listHoraires, getHoraire, createHoraire, updateHoraire, deleteHoraire	Gestion des voyages.	horaire enrichi (avec places_occupees) ; IN_USE s'il y a des réservations
getOccupiedSeats(id_horaire)	Numéros de places non annulées.	tableau trié
createReservation(data)	Réserver une place. **Atomique.**	réservation enrichie ; SEAT_TAKEN, DUPLICATE_TICKET
getReservationByTicket, getReservationById, listReservationsByUser, listReservations	Lire les réservations (par ticket, par voyageur, avec filtres).	réservations enrichies
cancelReservation(id), updateReservationStatus(id, statut)	Annuler (libère la place) ou changer le statut.	SEAT_TAKEN si on réactive une place reprise
expirePendingReservations(cutoffIso)	Annule les réservations « en attente » plus anciennes que la date donnée.	nombre annulé
createPayment, getPaymentByReference, getLatestPaymentByReservation, updatePayment	Enregistrer et suivre les paiements.	paiement ou null
getStats({today, id_compagnie})	Chiffres du tableau de bord.	{reservations_du_jour, trajets_actifs, taux_remplissage, utilisateurs}

5. Paiement en ligne : où mettre les clés

Le paiement est **optionnel** (cahier des charges). Il se choisit avec une seule ligne du `.env` : `PAYMENT_MODE`.

PAYMENT_MODE	Comportement	Quand l'utiliser
`off`	Pas de paiement. La réservation est confirmée tout de suite et le ticket est délivré.	Valeur par défaut. Démonstration du cœur du projet.
`simulation`	La réservation est « en attente » (place bloquée `RESERVATION_ATTENTE_MINUTES` minutes). Une page simule un paiement Mobile Money (MTN MoMo, Moov Money, Celtiis Cash). **Aucun argent n'est débité.**	Présentation devant le professeur, sans compte de paiement.
`fedapay`	Vrai paiement : redirection vers la page sécurisée FedaPay, puis confirmation après vérification serveur.	Quand un compte FedaPay existe. Commencer par `FEDAPAY_ENV=sandbox`.

5.1 Où mettre les clés FedaPay

Étape	Action
1	Créer un compte sur fedapay.com et ouvrir le tableau de bord (mode « test » / sandbox pour commencer).
2	Dans le tableau de bord, repérer la page des **clés API**. On y trouve une clé publique et une clé secrète. En général, les clés de test commencent par `pk_sandbox_` / `sk_sandbox_` et les clés réelles par `pk_live_` / `sk_live_` (à vérifier dans votre tableau de bord).
3	Ouvrir le fichier `.env` et remplir : `PAYMENT_MODE=fedapay`, `FEDAPAY_ENV=sandbox`, `FEDAPAY_PUBLIC_KEY=pk_...`, `FEDAPAY_SECRET_KEY=sk_...`.
4	Vérifier `APP_BASE_URL` : c'est l'adresse où FedaPay renvoie le voyageur après paiement (`APP_BASE_URL/paiement/retour`).
5	Relancer le serveur (`npm start`). La console affiche « Paiement : fedapay ».
6	Tester une réservation en mode sandbox avec les moyens de paiement de test indiqués par FedaPay. Vérifier : réservation « confirmée », paiement « reçu » sur le ticket, ligne dans la table `paiement`.
7	Pour passer en réel : faire valider le compte FedaPay, remplacer par les clés `live`, mettre `FEDAPAY_ENV=live` et utiliser une adresse HTTPS publique.

Où sont lues les clés : `.env` puis `src/config.js` (section `payment.fedapay`) puis `src/payments/fedapay.js`. La **clé secrète** ne doit jamais apparaître dans le code, dans GitHub, dans une capture d'écran ni dans une page web : elle reste uniquement dans le `.env` (ou dans les variables d'environnement de l'hébergeur).

5.2 Déroulement d'un paiement FedaPay

#	Qui	Action
1	Voyageur	Réserve une place : la réservation est « en attente » et la place est bloquée.
2	Navigateur	Page `paiement.html` : appelle `POST /api/paiements/initier`.
3	Serveur	Crée une transaction chez FedaPay (montant en FCFA = tarif du voyage) et demande l'adresse de la page de paiement. Enregistre un paiement « en attente ».
4	Voyageur	Est redirigé vers la page FedaPay et paie (Mobile Money ou carte).
5	FedaPay	Renvoie le voyageur sur `/paiement/retour?id=...`.
6	Serveur	**Interroge FedaPay** pour connaître le vrai statut (il ne croit jamais le navigateur). Si le paiement est approuvé : paiement « reçu », réservation « confirmée », e-mail du ticket.
7	Voyageur	Arrive sur son ticket. Si le paiement échoue, il peut réessayer tant que le délai d'attente n'est pas dépassé.

Cas particuliers gérés : délai dépassé (la place est libérée automatiquement) ; paiement reçu alors que la place a été reprise (message à l'écran et alerte dans la console : remboursement manuel) ; annulation d'une réservation déjà payée (le site signale qu'un remboursement est à traiter). **Important :** l'intégration FedaPay est écrite d'après la documentation publique et **n'a pas été testée avec un vrai compte** ; à valider en sandbox et à comparer avec la documentation officielle avant toute mise en ligne.

5.3 Autres prestataires (KkiaPay, etc.)

Les lignes `KKIAPAY_*` du `.env` sont réservées. Le fichier `src/payments/kkiapay.js` est un emplacement vide. KkiaPay fonctionne avec un « widget » à intégrer dans la page : il faudrait aussi autoriser son adresse dans la politique de sécurité (CSP) de `src/app.js` et vérifier la transaction côté serveur. Tout nouveau prestataire s'ajoute en créant un fichier dans `src/payments/` qui fournit `createCheckout` et `getStatus`, puis en l'appelant dans `src/services/paymentService.js`.

6. E-mail de confirmation (optionnel)

Si `SMTP_HOST` est vide, aucun e-mail n'est envoyé (le site fonctionne quand même). Pour un compte Gmail :

Étape	Action
1	Activer la validation en deux étapes sur le compte Google.
2	Créer un **mot de passe d'application** (rubrique Sécurité du compte Google). Ce mot de passe de 16 caractères remplace le mot de passe habituel.
3	Dans `.env` : `SMTP_HOST=smtp.gmail.com`, `SMTP_PORT=587`, `SMTP_SECURE=false`, `SMTP_USER=votre.adresse@gmail.com`, `SMTP_PASS=le mot de passe d'application`, `MAIL_FROM="Voyagéo "`.
4	Relancer le serveur. La console affiche « E-mail : activé ». Réserver une place : le voyageur reçoit son ticket par e-mail.

Une défaillance de l'envoi (erreur de mot de passe, panne) n'empêche jamais la réservation : elle est seulement signalée dans la console. Pour un vrai site, préférer un service d'envoi dédié (le code est le même : seuls les `SMTP_*` changent).

7. Routes de l'API

Toutes les requêtes qui modifient des données (POST, PUT, DELETE) doivent envoyer l'en-tête `X-CSRF-Token` (jeton obtenu par `GET /api/csrf`). Le frontend le fait automatiquement (`public/js/api.js`). Les erreurs ont la forme `{"erreur": "message", "code": "CODE"}`.

Méthode et adresse	Accès	Rôle
`GET /api/csrf`	Public	Jeton anti-CSRF.
`GET /api/config`	Public	Mode de paiement, délais.
`GET /api/villes`	Public	Liste des villes.
`GET /api/horaires?depart=&arrivee;=&date;=`	Public	Recherche de voyages.
`GET /api/horaires/:id`	Public	Détail d'un voyage et places occupées.
`GET /api/verifier/:numero`	Public	Vérification d'un ticket (lien du QR code), sans données personnelles complètes.
`GET /api/auth/moi`	Public	Utilisateur connecté ou null.
`POST /api/auth/inscription`	Public	Création de compte voyageur.
`POST /api/auth/connexion`	Public	Connexion.
`POST /api/auth/deconnexion`	Public	Déconnexion.
`POST /api/auth/mot-de-passe`	Connecté	Changer son mot de passe.
`POST /api/reservations`	Connecté	Réserver une place.
`GET /api/reservations/moi`	Connecté	Mes réservations.
`GET /api/reservations/:numero`	Propriétaire ou personnel autorisé	Ticket avec QR code.
`POST /api/reservations/:numero/annuler`	Propriétaire ou personnel autorisé	Annuler (24 h avant le départ pour un voyageur).
`POST /api/paiements/initier`	Connecté	Démarre un paiement FedaPay.
`POST /api/paiements/simuler`	Connecté	Paiement simulé (mode simulation).
`GET /paiement/retour`	Navigateur	Retour du prestataire de paiement.
`GET /admin`	Admin, gestionnaire	Page d'administration.
`GET /api/admin/stats`	Admin, gestionnaire	Statistiques (limitées à la compagnie du gestionnaire).
`GET|POST /api/admin/compagnies, PUT|DELETE .../:id`	Admin (lecture : gestionnaire)	Compagnies.
`GET|POST /api/admin/bus, PUT|DELETE .../:id`	Admin, gestionnaire (sa compagnie)	Bus.
`GET|POST /api/admin/trajets, PUT|DELETE .../:id`	Admin, gestionnaire (sa compagnie)	Trajets.
`GET|POST /api/admin/horaires, PUT|DELETE .../:id`	Admin, gestionnaire (sa compagnie)	Voyages. Passer un voyage en « annulé » annule ses réservations.
`GET /api/admin/reservations`	Admin, gestionnaire (sa compagnie)	Réservations (filtres statut, q, id_horaire).
`POST /api/admin/reservations/:id/annuler`	Admin, gestionnaire (sa compagnie)	Annuler une réservation.
`GET|POST /api/admin/utilisateurs, PUT .../:id`	Admin	Comptes : création, rôle, blocage, nouveau mot de passe.

8. Sécurité

Mesure	Détail	Fichier
Mots de passe	Hachés avec scrypt (intégré à Node.js) avec un sel aléatoire. Jamais stockés ni affichés en clair. Minimum 8 caractères.	`src/services/passwords.js`
Sessions	Cookie `HttpOnly`, `SameSite=Lax`, `Secure` en production. Le compte est revérifié à chaque requête : un compte bloqué est déconnecté immédiatement.	`src/app.js`, `src/middleware/auth.js`
Anti-CSRF	Jeton lié à la session, exigé sur toute requête qui modifie des données.	`src/middleware/csrf.js`
En-têtes et CSP	Helmet + Content-Security-Policy : aucun script ni style « inline », uniquement le site lui-même (et Google Fonts pour les polices).	`src/app.js`
Anti-XSS	Le frontend construit la page avec `createElement` et `textContent`, jamais `innerHTML` avec des données.	`public/js/ui.js`
Limitation des tentatives	20 tentatives de connexion/inscription par 15 minutes et par adresse IP (en production).	`src/routes/auth.js`
Validation	Chaque donnée reçue est vérifiée côté serveur (type, longueur, format), indépendamment du navigateur.	`src/utils/validate.js`
Droits d'accès	Rôles voyageur / gestionnaire / administrateur ; un gestionnaire ne voit et ne modifie que sa compagnie ; un voyageur ne voit que ses tickets.	`src/routes/admin.js`, `src/middleware/auth.js`
Ticket et QR code	Le QR code ne contient qu'un lien avec le numéro du ticket. La page de vérification n'affiche que « Prénom I. ».	`src/services/reservationService.js`
Identifiants	Le message d'échec de connexion est identique que l'e-mail existe ou non.	`src/services/authService.js`

9. Mise en ligne : liste de contrôle

- **Hébergement Node.js** (VPS ou plateforme d'hébergement d'applications). Les offres gratuites changent souvent : vérifier leurs conditions.
- Renseigner les variables du `.env` dans le panneau de l'hébergeur plutôt que d'envoyer le fichier : `NODE_ENV=production`, `SESSION_SECRET` (longue phrase aléatoire), `APP_BASE_URL=https://...`, `TRUST_PROXY=true`.
- **HTTPS obligatoire** (cookies sécurisés, paiement). La plupart des hébergeurs le fournissent.
- Installer avec `npm install --omit=dev` puis lancer `npm start` (idéalement avec un gestionnaire de processus qui relance le site en cas de plantage).
- **Sessions :** elles sont gardées en mémoire (perdues à chaque redémarrage, non adaptées à plusieurs serveurs). Pour un vrai site, brancher un « session store » (par exemple MySQL) dans `src/app.js`, paramètre `store` de `express-session`.
- Mettre `DEMO_DATA=false`, changer le mot de passe administrateur, utiliser une base de données réelle (pas « json »).
- Base de données : compte dédié aux droits limités, **sauvegardes régulières** (au moins une par semaine) stockées hors du serveur, restauration testée une fois.
- Paiement réel : clés `live`, compte FedaPay validé, test complet avec un petit montant.
- Informer les utilisateurs de l'usage de leurs données personnelles (nom, téléphone, pièce d'identité) et ne conserver que le nécessaire.

10. Tests et dépannage

La commande `npm test` exécute 23 tests automatiques : contrat de la base (12), API complète (8), paiement simulé et expiration (3). Le site a aussi été parcouru dans un vrai navigateur (inscription, recherche, choix de place, paiement simulé, ticket, administration, affichage mobile) sans erreur.

Symptôme	Cause probable	Solution
Cannot find module 'express'	Les dépendances ne sont pas installées.	`npm install`
La page n'a pas de style / Bootstrap introuvable	Bootstrap vient de `node_modules/`.	`npm install` puis relancer.
process.loadEnvFile is not a function	Version de Node.js trop ancienne.	Installer Node.js 20.12 ou plus récent.
ERREUR AU DÉMARRAGE : adaptateur incomplet	Des fonctions du contrat manquent dans l'adaptateur.	Le message liste les fonctions à écrire.
Le mot de passe administrateur est refusé	Caractère # sans guillemets dans le `.env`, ou mot de passe modifié depuis le site.	Mettre la valeur entre guillemets puis `npm run admin:reset`.
Port 3000 déjà utilisé	Un autre programme utilise ce port.	Changer `PORT` (et `APP_BASE_URL`).
« Session expirée ou requête non autorisée »	Jeton anti-CSRF périmé (session perdue).	Recharger la page.
Le QR code ou l'e-mail mène à localhost	`APP_BASE_URL` n'est pas à jour.	Mettre l'adresse publique du site.
Une réservation reste « en attente »	Paiement non terminé.	La place est libérée automatiquement après `RESERVATION_ATTENTE_MINUTES` minutes.
Polices non chargées hors connexion	Poppins et Open Sans viennent de Google Fonts.	Normal : le site utilise Arial à la place.
Erreur de connexion MySQL	Mauvais `DB_*`, service arrêté ou droits insuffisants.	`npm run check:db` affiche le détail.

Contacts du projet : Caleb (développement, Première DWM, HTC-CETIP) et son collègue (base de données).